

PA 3,4 - Lists, Stacks & Queues - Refining a Shortlist of Candidate Meals

Big idea: put Units 3 & 4 to work on the Menu Planner: **implement** a circular linked list, then **use** it (with a stack and a queue) to **grow and refine** a shortlist of candidate meals.

Today

- ▶ **implement** a circular linked list (adapt a plain one)
- ▶ **grow & refine** a shortlist: requests in order (**queue**), **undo** the latest (**stack**)
- ▶ **move** meals by relinking, never copying; **rotate** the finished list

The Project — One Class per File

Given (do not modify)

The whole menu model (`Course`, `Meal`, `MenuModel`, `mealScore`, ...), a *complete* plain `SinglyLinkedList` (the worked example you adapt), `Stack`, `Queue`, `SinglyLinkedListNode`, and `Request` — one request the queue carries, in three kinds (`add-meal`, `set-course`, `remove-lowest`) — each in its own header.

You submit TWO files

- ▶ `CircularSinglyLinkedList.hpp` — you **implement** the ring.
- ▶ `Shortlist.hpp` — you **design** the `Shortlist` class that uses it.

Your ring and the provided plain `SinglyLinkedList` share `SinglyLinkedListNode<Meal>` — that is what lets a meal **move** between them by relinking one node, with **no copy**.

The Core Idea — A Shortlist You Grow and Refine

The shortlist, and the requests that shape it

You keep a small **shortlist** of candidate meals — *not* every possible meal (far too many). It starts **empty**; three **queued requests** (arrival order) and two direct calls shape it:

- ▶ **add-meal** → insert it in score order, *unless* it disagrees with a course already fixed (then reject it);
- ▶ **set-course** → fix one course to one dish: drop the meals already there that pick a different dish, and reject later disagreeing add-meals;
- ▶ **remove-lowest** → remove the k lowest-scoring meals;
- ▶ **undo** → reverse the most recent *successful* request;
- ▶ **finalize** → no more changes, so **rotate** the list, featuring each in turn.

Move meals by relinking a node — copying one deep-copies its array; a move is just $O(1)$.

Part 1 — Make the List a Ring

Plain list

head $\rightarrow$ a $\rightarrow$ b $\rightarrow$ c $\rightarrow$ / (null)

The chain ends at nullptr, so the cursor stops when `curr == nullptr`.

Ring (you build)

`_rear->next == front`, so there is **no null** terminator. The front is `_rear->next`, the back is `_rear`, and the cursor is **count-bounded** (`atEnd()` compares its index to the length).

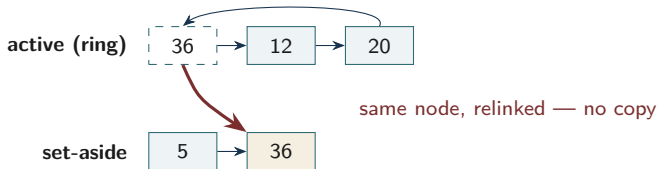
Singly linked, so the ends differ: *reaching* either is $O(1)$, but *removing* is cheap at only **one** of them — unlinking the other needs the node before it. Work out which as you build.

You fill the // TODOs: `front/back`, `rotate` (move `_rear` one hop, $O(1)$), `ring-closing`, `append/prepend`, `node-level detachAt/attachBack/attachBefore`, and the cursor's `atEnd()`. The plain `SinglyLinkedList` shows how to do each of these without the ring.

Move, Don't Copy — One Node Between Two Lists

The node is the shared currency

A Meal owns a heap array, so **copying** one is real work: a new array, deep-copied. Instead you **move** the node — `detachAt` unlinks it from the active ring, then `attachBack/attachBefore` relinks that *same* node into another list. This is $O(1)$, and nothing is copied.



The move check counts the meals `setCourse`, `removeLowest`, and `undoLast` deep-copy — it should be **0**. `addMeal` copies a meal by design and is not counted; undoing an add-meal *deletes* a node, which is not a copy either; `featuredMeal` returns a *reference*, so it never copies at all.

You DESIGN a class that owns the active list + undo history

1. `addMeal` $\rightarrow$ `bool` — insert the meal in score order and return `true`; but if it disagrees with a fixed course, reject it (`false`, nothing added).
2. `setCourse` $\rightarrow$ `bool` — fix a course to a dish; *already* set $\rightarrow$ `false`. Else move out the meals picking a different dish, block later violating add-meals, return `true`.
3. `removeLowest` — move out the k lowest-scoring meals; if k exceeds the size, remove **all**.
4. `applyRequests` — take each `Request` off the **queue** in arrival order and **dispatch** it by kind to 1–3.
5. `undoLast` — pop the undo **stack** and reverse the most recent *successful* request (what that takes depends on its kind — next slides).
6. `featuredMeal` + `rotate` — the spotlight meal *by reference*, then one hop.

The active list is your private `CircularSinglyLinkedList<Meal>`. The rest — set-course tracking, undo-record shape — is **your** design, graded on **outcome**, not representation.

You Own the Undo History

One record per successful request — and it is heterogeneous

Each *successful* request pushes one record onto your undo history (a **stack**); `undoLast` pops the top and reverses it. Only requests that *changed* something are undoable — a rejected add-meal, an already-set set-course, or a remove-lowest that removed nothing all record nothing. Since the three kinds reverse **differently**, one record must hold enough to reverse a request of *any* kind.

`undoLast` dispatches on the kind

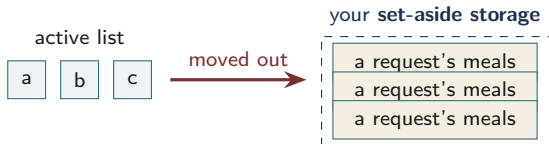
- ▶ an **add-meal** → find the meal it added and **destroy** it (no redo — it is gone for good);
- ▶ a **set-course** → **unset** the course *and* put its set-aside meals back;
- ▶ a **remove-lowest** → put its set-aside meals back.

That record, and how your history stores it, is **your** design — keep the fixed signatures and read-only view (`size/isEmpty/undoDepth/begin/end`). Owning the history means **freeing** it: a leak scores **0**.

Removing Doesn't Free — It Sets Aside

A set-course or remove-lowest MOVES meals out — it does not free them

When `setCourse` or `removeLowest` takes meals out, their nodes leave the **active list** but are **not freed** — they are **moved** (relinked) into your **set-aside storage**, wherever you keep it. The `Shortlist` **holds** them there so undo can move them back, until that request is undone or the `Shortlist` is destroyed.



*How you set them aside is your design — e.g. a separate list per request, or an array — wherever you choose. (In the kitchen they wait on a **numbered tray**, each pointed to by its **position**.) Whatever you allocate to hold them the `Shortlist` **owns**; free it, or it leaks — a leak scores **0**.*

Set-Aside Meals — Restored or Freed

Each set-aside meal ends up in one of two places

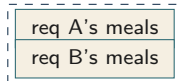
A meal that `setCourse` or `removeLowest` set aside ends up exactly one of two ways:

1. **undoLast** — **moves** it back into the active list (relink, no copy), and frees the storage that held that request's meals.
2. **never undone** — it waits in your set-aside storage until the `Shortlist` is destroyed, when **your destructor** frees that storage *and* the meals inside it.

You delete the **container** and the meals ride out with it — you never hand-delete each meal (the one exception is undoing an **add-meal**, which deletes that single node).

The grader leaves set-aside meals un-undone, then destroys the `Shortlist` (here: one `undoLast`, 2 still held). Free every pending record *and* your set-course bookkeeping:

both still held at destruction



applyRequests — a queue (FIFO)

Take each request off the queue **in arrival order** and **dispatch** it by kind; each *successful* one pushes a record onto the undo **stack** — whatever it takes to reverse it.

undoLast — a stack (LIFO)

pop the **top** (the most recent successful request) and **reverse** it — move its set-aside meals back into the active list, or *destroy* the meal an add-meal put in.

A queue is right because requests are honored **in order**; a stack is right because you can only undo the **most recent** change.

Where You Keep Things Matters

The same meals, over and over

You choose the order your list keeps. Some meals are removed and restored many times over a session — and what those repeated changes cost depends **entirely** on that choice. And diners always ask about the top pick — keep the **first prioritized meal** (the **highest**-scoring) easy to reach in $O(1)$.

Part of your grade rewards a good ordering — we do not tell you which. That reasoning is yours.

rotate() — the finale

Once no more meals will be removed, `rotate()` advances the ring one hop in $O(1)$ — so you can feature each meal in turn without moving or reallocating anything in the shortlist. (Recording the featured meal does copy it; that copy is by design and is not counted by the move check.)

Rules and How It's Graded

Rules

- ▶ Submit `CircularSinglyLinkedList.hpp` + `Shortlist.hpp` only (`Request.hpp` is provided).
- ▶ **No** STL containers / smart pointers / library sort-search; include only `<utility>` `<string>` `<fstream>` `<stdexcept>` — **not** `<iostream>`; no `using namespace`. (All enforced.)
- ▶ In the ring file define **no node type of your own** — only the class and its `Cursor` / `Iterator`; helper *methods* are fine.
- ▶ **Move, don't copy** (graded — read meals as `Meal const&`); keep the ring closed; no leaks / double-frees.

Grading (50 pts)

- ▶ **Memory safety first** — any leak scores **0**.
- ▶ Circular-list conformance — **8** (build / rotate / detach / attach).
- ▶ Shortlist request behavior — **8** (add-meal / set-course / remove-lowest / undo).
- ▶ Model results — **18** (list contents, **order-independent**).
- ▶ Moves, not copies — **8** (`setCourse` / `removeLowest` / undo relink).
- ▶ A large removal + its undo — **4** (measured).
- ▶ Performance — **4** (your ordering).

Every correctness test is fully shown; some are freshly randomized. Know your own code.

Takeaways

- ▶ You **implemented** a ring by adapting a plain linked list — count-bounded cursor, closed loop.
- ▶ A linked node **moves** between lists by relinking: no copy, no shift.
- ▶ A **queue** applies requests in order; a **stack** undoes the most recent — and undo is **heterogeneous**: restore the set-aside meals, or discard an added one.
- ▶ You **own** your undo history and set-aside meals — undo them or your destructor frees them; a leak scores **0**.

Class Question

Which order did you keep — what does each repeated operation cost under it, and *why*? And why does the ring's front behave differently from its back?